

Análise Comparativa dos Algoritmos de Busca em Profundidade e Busca em Largura Aplicadas ao Problema do Labirinto

**Gustavo Augusto Costa, Gabriel Alvisé Schenorberger, João Gabriel Gnoatto,
Pedro Henrique Honório**

Engenharia de Software – Centro Universitário da Fundação Assis Gurgacz (FAG)
Caixa Postal 85806-095 – Cascavel – PR – Brasil

Resumo. Este trabalho apresenta a implementação e análise comparativa dos algoritmos de Busca em Profundidade (Depth-First Search - DFS) e Busca em Largura (Breadth-First Search - BFS) aplicados à resolução do Problema do Labirinto. O problema é representado por uma matriz de posições, permitindo a exploração de diferentes estados até a identificação de um caminho entre a posição inicial e a saída. Os algoritmos foram avaliados considerando tempo de execução, consumo de memória, quantidade de posições examinadas e quantidade de movimentos do caminho encontrado. Os experimentos foram realizados no mesmo ambiente computacional, utilizando dez amostras para a medição do tempo, com 1.000 execuções em cada amostra, e dez execuções individuais para a medição de memória. A partir dos resultados obtidos, foi realizada uma comparação entre as duas estratégias de busca.

1. Introdução

Os algoritmos de busca são importantes na área de Inteligência Artificial, pois permitem explorar diferentes possibilidades para encontrar uma solução para determinado problema. Entre as estratégias de busca existentes, destacam-se a Busca em Profundidade (Depth-First Search - DFS) e a Busca em Largura (Breadth-First Search - BFS), que utilizam diferentes formas de exploração dos estados durante a busca.

Neste trabalho, os algoritmos de Busca em Profundidade e Busca em Largura são aplicados ao Problema do Labirinto. O problema consiste em representar o labirinto por meio de uma estrutura de estados, na qual determinadas posições podem ser percorridas e outras não, buscando encontrar um caminho entre uma posição inicial e uma posição final.

A partir dessa representação, os dois algoritmos foram implementados e utilizados para solucionar o problema proposto. A aplicação dos algoritmos permite observar como cada estratégia realiza a exploração das posições disponíveis e como o processo de busca é conduzido até que uma solução seja encontrada.

O objetivo deste trabalho é implementar os algoritmos DFS e BFS para a resolução do Problema do Labirinto e realizar uma análise comparativa entre as duas estratégias. A comparação considera aspectos como tempo de execução, consumo de memória, quantidade de posições examinadas e quantidade de movimentos do caminho encontrado.

2. Formulação do problema

O problema foi interpretado como uma busca em uma matriz que representa um labirinto. Cada posição da matriz corresponde a um estado que pode ser explorado pelos algoritmos de busca. O objetivo é encontrar um caminho entre a posição inicial e a posição final, passando somente por posições livres.

2.1 Representação do labirinto

O labirinto utilizado possui cinco linhas e cinco colunas. Na representação adotada no código, o valor (1) representa uma posição livre, enquanto o valor (0) representa uma parede. Dessa forma, os algoritmos podem percorrer somente as posições que possuem valor (1).

A matriz utilizada é:

```
1 0 0 0 0
1 1 1 0 1
0 0 1 0 1
0 1 1 1 1
0 0 0 0 1
```

2.2 Estado Inicial

O estado inicial é representado pela posição (0, 0), correspondente à primeira linha e à primeira coluna da matriz. Essa posição é utilizada como ponto de partida para as duas buscas.

2.3 Estado Final

O estado final é representado pela posição (4, 4), correspondente à última linha e à última coluna da matriz. A busca termina quando essa posição é alcançada.

2.4 Movimentos Possíveis

A partir de cada posição, são consideradas quatro possibilidades de movimento: cima, baixo, esquerda e direita. Movimentos diagonais não são considerados. Além disso, uma nova posição somente pode ser utilizada quando está dentro dos limites da matriz e corresponde a uma posição livre.

2.5 Busca em Profundidade - DFS

A Busca em Profundidade utiliza uma pilha para armazenar as posições que ainda serão exploradas. A implementação inicia pela posição inicial e retira da pilha a última posição adicionada. Quando uma posição é visitada, ela é registrada como explorada e seus vizinhos disponíveis são adicionados à pilha. O processo continua até que a posição final seja encontrada ou não existam mais posições para explorar. A implementação também mantém o caminho percorrido e registra a quantidade de posições examinadas.

Algoritmo DFS

criar pilha com a posição inicial

criar conjunto de visitados

enquanto a pilha não estiver vazia:

 retirar o último elemento da pilha

 se for a posição final:

 retornar caminho

 marcar posição como visitada

 para cada vizinho não visitado:

 adicionar vizinho à pilha

2.6 Busca em Largura - BFS

A Busca em Largura utiliza uma fila para organizar as posições que serão exploradas. Dessa forma, as posições são analisadas seguindo a ordem em que são adicionadas à fila. Assim como na DFS, a implementação mantém as posições visitadas, registra a quantidade de posições exploradas e armazena o caminho percorrido até a posição final.

Algoritmo BFS

criar fila com a posição inicial

marcar posição inicial como visitada

enquanto a fila não estiver vazia:

 retirar o primeiro elemento da fila

 se for a posição final:

 retornar caminho

 para cada vizinho não visitado:

 marcar vizinho como visitado

 adicionar vizinho à fila

3. Resultados dos Experimentos

3.1 Ambiente de Execução

Os experimentos foram realizados em um computador equipado com processador Intel(R) Core(TM) i7-1255U e 40 GB de memória RAM, utilizando o sistema operacional Windows 11. A implementação foi executada utilizando a linguagem Python, versão 3.12.4. Para a execução dos algoritmos foram utilizadas somente bibliotecas nativas do Python, sem dependência de bibliotecas externas.

Os dois algoritmos foram executados no mesmo ambiente e utilizando o mesmo labirinto, mantendo as mesmas condições de execução para possibilitar uma comparação entre os resultados obtidos.

3.2 Metodologia dos Experimentos

Para comparar o desempenho dos algoritmos DFS e BFS, foram realizadas dez medições para cada algoritmo. Para a análise do tempo de execução, cada medição foi baseada em 1.000 execuções do respectivo algoritmo. O código utiliza `perf_counter_ns()` para realizar a medição do tempo e converte o resultado para milissegundos por execução.

Após a coleta das dez amostras, foram descartados o maior e o menor valor de cada métrica. Os oito valores restantes foram utilizados para calcular a média e o desvio padrão.

O consumo de memória foi medido separadamente das medições de tempo. Para cada uma das dez medições de memória foi realizada uma execução do algoritmo, utilizando `tracemalloc` para obter o pico de memória observado durante a busca. Os valores foram convertidos para KiB.

Essa metodologia foi aplicada aos dois algoritmos nas mesmas condições experimentais.

Métrica	DFS	BFS
Tempo médio	0,0080055 ms	0,009720625 ms
Desvio padrão do tempo	0,000281989 ms	0,000339590 ms
Memória média	1,086 KiB	2,117 KiB
Desvio padrão da memória	0 KiB	0 KiB
Posições examinadas	9	11
Movimentos do caminho	8	8
Encontrou a saída?	sim	sim

3.3 Análise do Tempo de Execução

Nos experimentos realizados, o tempo médio de execução da DFS foi de 0,0080055 ms, enquanto o da BFS foi de 0,009720625 ms. O desvio padrão observado foi de 0,000281989 ms para a DFS e de 0,000339590 ms para a BFS.

Considerando especificamente o labirinto utilizado e as condições do experimento, a DFS apresentou menor tempo médio de execução que a BFS. Esses valores representam o comportamento observado na implementação analisada e não devem ser generalizados para todos os problemas de busca, pois o desempenho pode variar conforme o tamanho e a estrutura do espaço de estados.

3.4 Análise do Consumo de Memória

Em relação ao consumo de memória, a DFS apresentou média de 1,086 KiB, enquanto a BFS apresentou média de 2,117 KiB. Nos dados coletados, o desvio padrão do consumo de memória foi de 0 KiB para os dois algoritmos.

O desvio padrão do consumo de memória foi igual a 0 KiB porque, nas medições realizadas, cada algoritmo apresentou o mesmo pico de memória em todas as execuções consideradas.

No experimento realizado, a BFS apresentou consumo médio de memória maior que a DFS. Essa diferença corresponde ao comportamento observado nas implementações utilizadas durante a exploração do labirinto.

3.5 Posições Examinadas e Caminho Encontrado

Durante a resolução do labirinto, a DFS examinou 9 posições, enquanto a BFS examinou 11 posições. Os dois algoritmos encontraram a saída do labirinto.

O caminho encontrado pelos dois algoritmos apresentou 8 movimentos. Portanto, neste experimento, a diferença entre os algoritmos ocorreu principalmente na quantidade de posições examinadas durante a busca, enquanto a quantidade de movimentos do caminho encontrado foi igual para ambos.

3.6 Comparação dos Algoritmos

Os resultados mostraram que ambos os algoritmos encontraram a saída do labirinto com um caminho de 8 movimentos. A DFS apresentou menor tempo médio de execução, menor consumo de memória e examinou menos posições que a BFS.

A DFS explora um caminho em profundidade antes de testar outras possibilidades, podendo utilizar menos memória, mas não garante encontrar o menor caminho. Já a BFS explora os estados por níveis e, quando todos os movimentos possuem o mesmo custo, encontra um caminho com a menor quantidade de movimentos, embora possa consumir mais memória.

Dessa forma, os dois algoritmos se mostraram adequados para o problema proposto. No labirinto utilizado, a DFS apresentou melhor desempenho nas métricas analisadas, mas esse resultado depende das características específicas do problema.

4. Conclusão

A implementação dos algoritmos de Busca em Profundidade (DFS) e Busca em Largura (BFS) permitiu comparar diferentes estratégias de exploração aplicadas ao Problema do Labirinto. Ambos os algoritmos encontraram a saída e produziram um caminho com 8 movimentos.

Nos experimentos realizados, a DFS apresentou menor tempo médio de execução, menor consumo de memória e examinou menos posições que a BFS. Apesar disso, os resultados estão relacionados ao labirinto e às condições utilizadas no experimento, não podendo ser generalizados para outros problemas.

Apesar de os dois algoritmos terem encontrado um caminho de 8 movimentos neste labirinto, isso não significa que a DFS sempre encontre o menor caminho. A BFS possui essa propriedade quando todos os movimentos apresentam o mesmo custo.

Dessa forma, os dois algoritmos se mostraram adequados para a resolução do labirinto proposto, apresentando comportamentos diferentes quanto à exploração dos estados e ao uso de recursos computacionais.

5. Referências

GASCHENORBERGER, Gabriel Alvise Schenorberger et al. *BuscaCegaIA: Projeto Acadêmico — Algoritmos de Busca*. GitHub, 2026. Disponível em: <https://github.com/gaschenorberger/BuscaCegaIA>. Acesso em: 14 set. 2026.

CODECADEMY. Breadth-First Search vs Depth-First Search: Key Differences. Codecademy, 2026. Disponível em: <https://www.codecademy.com/article/bfs-vs-dfs>. Acesso em: 14 set. 2026.